

Ejemplo 4: Crear un Post con Fetch API y Promesas (POST)

Objetivo del ejercicio

- Aprender a **enviar datos a un servidor** usando `fetch()` con método POST.
- Practicar la creación y manejo de **promesas con `then`, `catch` y `finally`**.
- Mostrar la respuesta del servidor al alumno en pantalla.

Código completo (HTML + Bootstrap + JS)

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ejemplo 4 - Fetch POST Promesas</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.
min.css" rel="stylesheet">
</head>
<body class="container py-4">

  <h1 class="mb-4">Ejemplo 4: Crear un Post con Fetch POST</h1>
  <p>Rellena los campos y envía un post a la API de prueba
<code>JSONPlaceholder</code>.</p>

  <form id="formPost" class="mb-3">
    <div class="mb-3">
      <label for="userId" class="form-label">User ID</label>
      <input type="number" class="form-control" id="userId"
value="1" required>
    </div>
    <div class="mb-3">
      <label for="titulo" class="form-label">Título del
Post</label>
      <input type="text" class="form-control" id="titulo"
placeholder="Título" required>
    </div>
    <div class="mb-3">
      <label for="contenido" class="form-
label">Contenido</label>
      <textarea class="form-control" id="contenido" rows="3"
placeholder="Escribe el contenido..." required></textarea>
    </div>
    <button type="submit" class="btn btn-primary">Enviar
Post</button>
  </form>

  <div id="resultadoPost" class="mt-3"></div>

  <script>
    const formPost = document.getElementById('formPost');
    const resultadoPost =
document.getElementById('resultadoPost');
```

```

formPost.addEventListener('submit', (e) => {
    e.preventDefault(); // Evitar recarga de página

    // Obtener valores del formulario
    const userId = document.getElementById('userId').value;
    const title = document.getElementById('titulo').value;
    const body = document.getElementById('contenido').value;

    // Mostrar mensaje de envío
    resultadoPost.innerHTML = `<div class="alert alert-
info">Enviando post...</div>`;

    // Petición fetch POST
    fetch('https://jsonplaceholder.typicode.com/posts', {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json'
        },
        body: JSON.stringify({
            userId: userId,
            title: title,
            body: body
        })
    })
    .then(response => {
        if (!response.ok) throw new Error(`Error HTTP:
${response.status}`);
        return response.json();
    })
    .then(data => {
        resultadoPost.innerHTML = `
        <div class="alert alert-success">
            Post creado con éxito!<br>
            ID: ${data.id}<br>
            Título: ${data.title}<br>
            Contenido: ${data.body}<br>
            User ID: ${data.userId}
        </div>`;
    })
    .catch(error => {
        resultadoPost.innerHTML = `<div class="alert alert-
danger">Error: ${error.message}</div>`;
    })
    .finally(() => {
        console.log("Petición POST finalizada");
    });
});
</script>

</body>
</html>

```

Explicación paso a paso

1. **Formulario HTML:** Permite al alumno ingresar `userId`, `title` y `body` del post.
2. **fetch() con método POST:**

- `method: 'POST'` indica que se enviarán datos al servidor.
 - `headers: {'Content-Type': 'application/json'}` define que se envía JSON.
 - `body: JSON.stringify({...})` convierte los datos a formato JSON.
3. **`then(response => response.json())`**: Convierte la respuesta del servidor a objeto JS.
 4. **`then(data => ...)`**: Muestra la respuesta, incluyendo el `id` generado por la API.
 5. **`catch(error => ...)`**: Maneja cualquier error en la petición.
 6. **`finally()`**: Se ejecuta siempre, para log o limpieza.
-

Preguntas de comprensión para alumnos

1. ¿Cuál es la diferencia entre `fetch` GET y POST?
 2. ¿Qué hace `JSON.stringify` y por qué es necesario?
 3. ¿Qué información devuelve la API después de crear un post?
 4. ¿Qué pasaría si no ponemos `Content-Type: application/json` en los headers?
 5. ¿Cómo podrías validar que los campos del formulario no estén vacíos antes de enviar?
-

Mejoras sugeridas para el alumno

1. Mostrar un **spinner de carga** mientras se envía el post.
2. Limpiar el formulario automáticamente al enviar correctamente.
3. Añadir validaciones avanzadas de los campos del formulario.
4. Guardar los posts enviados en un **array local** y mostrarlos en la lista sin recargar.
5. Enviar posts a otra API de prueba o backend local usando Node.js y Express.